BANZA: An Open Protocol for Financial Interoperability

Fidel R. Monteiro[1] and Jesus R. Monteiro[1]

[1]BANZAMI – Tecnologia e Serviços, Lda., Luanda, Angola
Official website: https://banza.network

Correspondence: Fidel R. Monteiro (contact@banza.network)

Abstract. Independent financial operators interoperate through bilateral integrations and shared infrastructures, but the specifications, the tests and the results are not always public, making technical validation hard to reproduce. This paper presents BANZA, an open financial interoperability protocol resting on public and versioned rules, on the separation between specification and implementation, on deterministic validation and on verifiable evidence. Its design follows four Fundamental Principles — *Robust*, *Resilient*, *Secure* and *Simple* —, from which it follows that where trust, authorisation, integrity or correctness cannot be established, the correct response is to refuse. The protocol defines contracts, schemas, L0–L4 conformance profiles and common mechanisms that allow implementations to be evaluated without conformance depending on the reference implementation, and keeps the planes of specification, technical conformance, certification, scheme operation and regulatory authorisation distinct. The normative surface of each version is explicitly indexed, signed or digested artifacts have a deterministic canonical representation, and the execution semantics — statuses, reason codes and idempotency — is public and accompanied by conformance vectors. The trust root rests on three authorities with a threshold of two, a pinned genesis and succession authorised by the predecessor set, with no path to unilateral recovery. Each result is bound to the inputs that produced it by canonical artifacts, cryptographic digests, reason codes and receipts, so that the technical basis of every evaluation can be inspected and reproduced by third parties. The protocol and the reference implementation remain in pre-production; independent third-party implementation, performance, scalability, adoption and behaviour in real operational environments have not yet been experimentally demonstrated.

1 Introduction

Financial operators rarely work in isolation. To transfer value or exchange messages, their implementations must agree on formats, identity, keys, error semantics and testing procedures. In bilateral integrations these elements are defined between each pair; when specifications, tests and results remain private, work is repeated and validation becomes difficult for third parties to reproduce.

Let n be the number of operators. Equations (1a) and (1b) represent, respectively, a mesh of bilateral integrations and the adoption of a common protocol.

The Portuguese-language edition is the canonical version of the BANZA Whitepaper. The English-language edition is an official translation. In the event of an unintended divergence, the canonical Portuguese edition prevails.

$$R_{\text{bilateral}}(n) = \frac{n(n-1)}{2} \tag{1a}$$

$$I_{\text{common}}(n) = n \tag{1b}$$

Equation (1a) counts pairwise relations in a complete bilateral mesh — an illustrative case, not a universal model — and Equation (1b) counts implementations when each operator adopts the same specification: one more operator requires n new integrations in the first regime and one in the second.

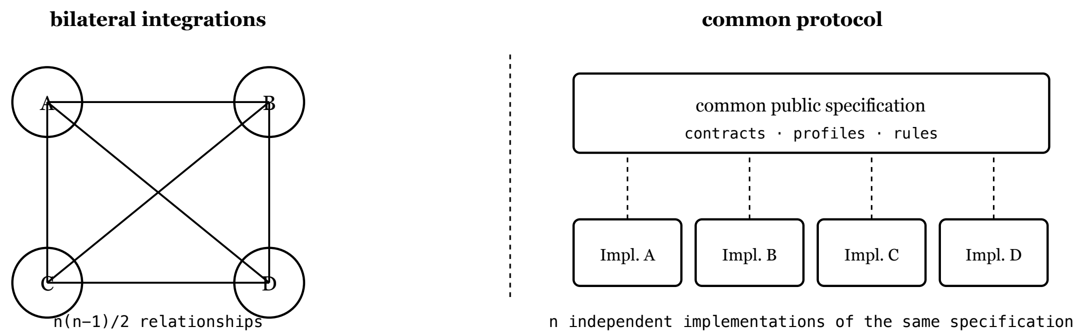

Figure 1. Complete bilateral mesh and common protocol. In the second regime the shared element is the public specification, not a central infrastructure.

A common protocol does not imply a common infrastructure. Each operator keeps its own implementation, infrastructure and operational relationships, provided it observes the specified behaviour and publishes the artifacts required by the version and profile it adopts. Interoperability may also rest on shared infrastructures, which continue to perform routing, clearing or settlement; BANZA does not replace them, it adds an open layer of specifications, profiles, versioned contracts, deterministic conformance and verifiable evidence.

Standards such as ISO 20022 [1], ISO 8583 [2], the EMV specifications for QR payments [3] and ISO/IEC 9646 [4] address components of messaging, payments and conformance testing. Mojaloop [9] is closer: it combines an interoperability specification between providers with an open-source platform of central services used to run schemes. The difference lies in how the authority of the specification, the implementation, the conformance evidence and the operational scheme are organised. BANZA complements them by defining an open protocol in which each implementation publishes at its canonical origin and is evaluated against a public version and profile — about implementations, not about entities.

This document is descriptive and non-normative.

2 Model

BANZA distinguishes an operator — the responsible entity — from an implementation, the technical system being evaluated. One operator may publish several, and any result applies to a determinate implementation, version, profile, environment, scope, evidence set and validity period, never to an entity in the abstract. Each implementation has a stable identifier, and the exact set of artifacts evaluated is fixed by a content digest: a different build is a distinct subject and requires its own evaluation.

We model an implementation as a six-element tuple. In Equation (2), o identifies the operator, i the implementation, v the protocol version, p the conformance profile, e the environment and u the canonical origin.

$$\mathcal{I} = (o, i, v, p, e, u) \tag{2}$$

The language, the database and the origin of the code are not part of this tuple: conformance depends on the contracts and on observable behaviour. The artifacts observed at the origin at an instant t form the set of Equation (3).

$$A_t(\mathcal{I}) = \{\, a_1, a_2, \ldots, a_n \,\} \tag{3}$$

The normative surface is not implicit: it is identified by the *BANZA Normative Manifest*, a versioned index declaring which artifacts define requirements — determining what must be implemented is a lookup in published material, not an inference from the code. Validation takes those artifacts and the specification S applicable to v and p; engine version m produces the result R — verdict and reason codes —, the evidence E and the receipts P.

$$V_m(A_t(\mathcal{I}), S_{v,p}) \to (R, E, P) \tag{4}$$

Scope is explicit: a result holds only for the combination it declares and is not extrapolated to the entity that published the artifact. Reproducibility is central: given equivalent canonical inputs, the same specification, profile and engine version, independent executions produce semantically equivalent verdicts and reason codes. The motivation is that of reproducible research [8], but the equivalence is not an analogy — the protocol defines it normatively, requiring equality of the decisional statuses, of the bindings to the observed inputs and of the core reason codes. Non-deterministic metadata is excluded, and byte equality is neither required nor used as the criterion.

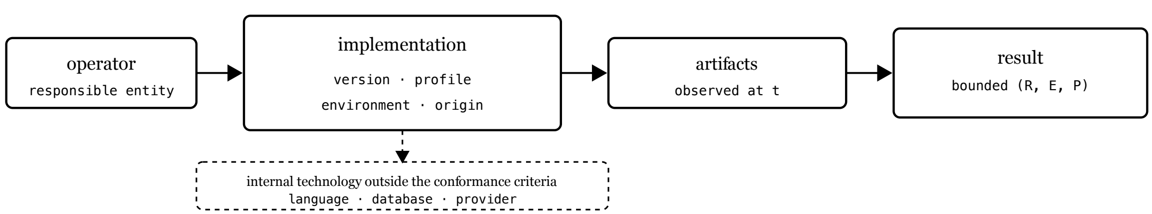

Figure 2. The result is bounded by the implementation and by the artifacts observed.

Content digests, computed with SHA-256 [5], identify exactly the inputs observed and make a validation claim re-evaluable by third parties. Comparing them requires equivalent representations to produce the same bytes. For JSON artifacts that are signed or digested, BANZA defines *BANZA Canonical JSON* (BCJ/1), a restricted profile of the JSON Canonicalization Scheme [10]: a deterministic UTF-8 representation, members ordered by UTF-16 code units, integers within the safe domain $\pm(2^{53} - 1)$, rejection of duplicate members, and no verifier-side Unicode normalization. Rejection precedes semantic interpretation: a parser that applies the first or the last occurrence has already lost the distinction the signature was meant to cover. Members not described by the schema are preserved and canonicalized like any other, and are covered by the signature or by the applicable digest.

3 Architecture

BANZA is organised in three institutional layers, separated by responsibility, infrastructure and keys. Layer 1 — Open protocol brings together specifications, contracts, profiles and common mechanisms for discovery, identity, trust, revocation and federation. Layer 2 — Conformance and Interoperability Certification evaluates implementations against public profiles and versions, with deterministic engines and verifiable evidence. Layer 3 — Independent operational schemes comprises infrastructures and networks that adopt BANZA, under their own legal, commercial and regulatory rules.

Design decisions follow four Fundamental Principles, named **BANZA $\mathbf{R^2S^2}$** — *Robust*, *Resilient*, *Secure* and *Simple* —, where R^2 stands for the two principles beginning with R and S^2 for the two beginning with S. *Robust* is to remain correct and deterministic under independent implementation, adversarial input and boundary conditions. *Resilient* is to contain failures, preserve safe operation where possible and recover deterministically without weakening guarantees — a safe refusal may be the resilient behaviour. *Secure* is to enforce critical properties by construction and fail closed when they cannot be established. *Simple* is to use the smallest mechanism sufficient for the required property. They are decision criteria, not desirable qualities: resilience does not override security, nor promise continuous availability.

The principles guide; the eight *Protocol Structural Properties* of the Reference characterise the protocol — among them failing closed, associated with *secure* and *resilient*, and not a fifth principle; the normative requirements define conformance; the evidence supports what may be claimed. Between them, five Architectural Invariants constrain the admissible architecture: *open specification* — the applicable rules, contracts and profiles are public and versioned; *implementation independence* — no particular implementation constitutes the protocol, and the reference implementation realises it without defining it; *independent verification* — the published artifacts suffice for a third party to assess the applicable conditions without depending on a private decision; *operational independence* — no central infrastructure is required to carry messages or funds; and *separation of decisions* — conformance, technical certification, admission to a scheme, operational agreements and regulatory authorisation are distinct processes.

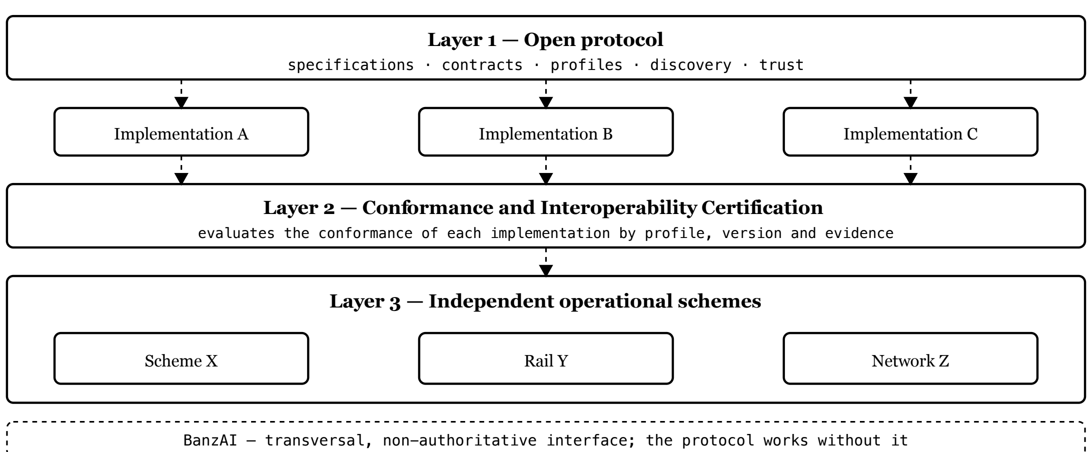

Figure 3. Layer 1 defines the specification, Layer 2 evaluates conformance through evidence, and Layer 3 remains operationally independent.

Technical adoption consists of implementing the public specification and satisfying the applicable contracts. The BanzAI is a human-facing interface for consultation and explanation, without taking part in the technical determination, and its unavailability blocks no protocol operation. The L0–L4 profiles of Section 4 are conformance levels, distinct from the three institutional layers.

4 Profiles

BANZA defines five cumulative levels, from L0 to L4. Adoption proceeds through the choice of a profile, the publication of artifacts at the canonical origin and the validation journey; a positive result evidences the technical conditions of the level, without being certification, authorisation or admission. Levels L0 to L2 can be assessed within a single operator.

L0 — Protocol Sandbox validates safe configuration in a test environment, with a valid manifest and correct monetary representation. L1 — Core Payment Capability adds essential payment and traceability capabilities. L2 — Payment Initiation Capability adds initiation by request or dynamic QR code. L3 — Inter-Operator Interoperability adds requirements and evidence of interoperability between operators — routing, settlement and reconciliation, functions performed by the applicable operators or schemes — and constitutes the federation threshold.

The capabilities each profile requires are identified by a common normative vocabulary, and satisfaction is decided by exact comparison of the published identifiers, without implicit normalization; only normatively registered *aliases* are accepted. The public vectors have a scope determined by the profile: the association between case and level is declared, not inferred.

L4 — External Interoperability inherits L3 and adds no universal normative artifacts of its own: its requirements and evidence come from an external-interoperability profile identified by identifier and version. Satisfying L3 does not satisfy L4 — with no profile selected the level remains unevaluated, not approved — and no external profile is published.

A certification profile is distinct from a level: it fixes a level and the capabilities, contracts and endpoints required, and is immutable once published.

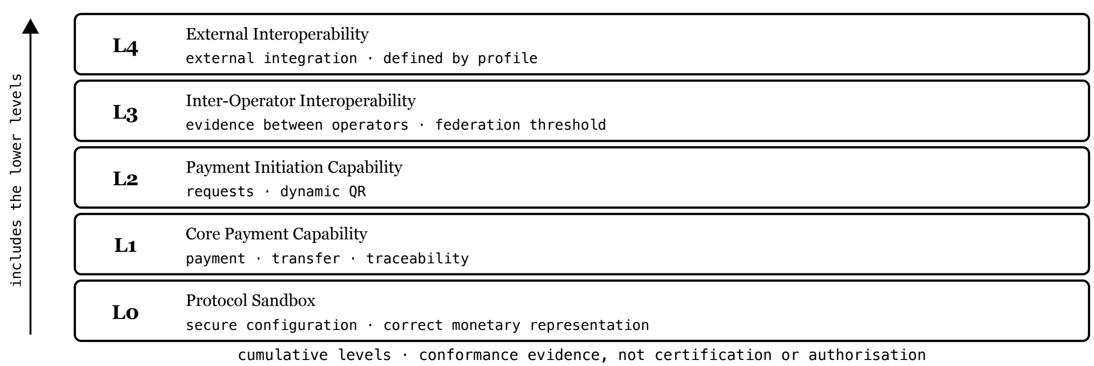

Figure 4. The profiles are cumulative; L3 adds the federation threshold and L4 depends on the selected external profile.

5 Discovery

Each implementation publishes its artifacts at a canonical origin controlled by the operator [7]. Discovery begins at a fixed path under that origin and returns what is needed to identify the implementation, the version, the profile, the endpoints, the signed metadata and the public keys.

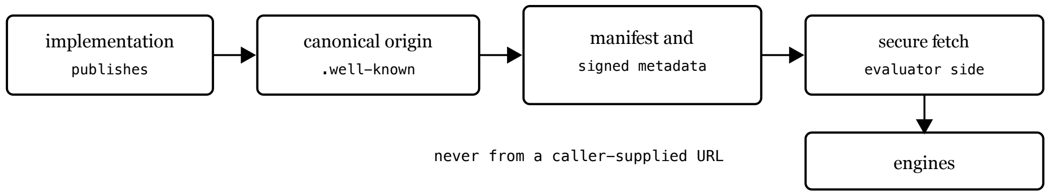

Figure 5. The implementation publishes at its canonical origin; the evaluator obtains the artifacts through a secure-fetch module, depending on no private decision.

Identity and integrity rest on published keys and signed metadata. The trust root is not a key: it is a set of three distinct authorities whose signing keys are held outside the serving infrastructure. Any authorisation exercised by the set requires two distinct authorities — a threshold that applies to what the root authorises, not to every protocol operation. An authority's identity is cryptographic and no organisation name takes part in validity: that the holders occupy independent control domains is a production governance requirement, to be evidenced before the first production ceremony.

The set advances as a lineage. The genesis set has sequence zero and no predecessor, and is accepted only when its digest equals one the verifier was explicitly configured with — trust on first use is refused. Every later set advances the sequence by one, names its predecessor by digest, and is authorised by two distinct authorities *of the predecessor set*, verified under the keys the predecessor records: the candidate does not authorise itself. Continuity follows — if an authority is lost, compromised or obstructive, the remaining two authorise a successor that replaces it without its participation, since requiring it would make the path three-of-three and grant it a veto over its own replacement. Below the threshold, canonical continuity is blocked: there is no emergency master key and no single-party path. The active set signs the Key Manifest with Ed25519 [6], which authorises delegated keys separated by domain; revocation and validity reject revoked or expired material.

Validity alone does not distinguish the current version from an earlier one still inside its period. For each class of trust material the protocol fixes a monotonic scope, a signed *ordering marker* and the state already accepted in that scope, kept per logical trust object rather than globally. The marker is whatever the class declares: a sequence, in the Root Authority Set; the instant at which the version comes into force, in dated artifacts. The rule is common — a lower marker is rejected, a higher and valid one becomes the accepted state, and at the same marker the authenticated content decides: an equal digest is an idempotent replay, a different one is equivocation, which fails closed and leaves the state unchanged. That distinction is necessary because two consecutive publications may legitimately share a time marker. Nothing depends on the order of retrieval: transport does not define trust, which comes from the authenticated material and the lineage that authorises it. The property is local — it protects against rollback and detects conflict at the same marker, without establishing a verifiable public history, global transparency or consistency across observers, and without detecting a divergent presentation (*split view*).

6 Validation

Validation follows a fixed journey of eight technical steps and one aggregation step, executed by deterministic engines. Each step ends in one of four terminal statuses — verified, pending, failed or blocked —, and no transient status is sealed into a receipt. Evaluation is fail-closed: absent, expired or inconsistent evidence never produces approval, and certification readiness requires every step of the profile verified.

The separation between determination and explanation also exists within the result: the status determines the technical outcome, the reason code explains it. A conformant consumer derives its behaviour from the status; a code it does not recognise is preserved and recorded, never changes a verdict and is not by itself grounds for rejection, and extension codes, in a namespace of their own, are never interpreted. The explanatory vocabulary can thus be extended without changing decisions, and because statuses and codes are public, independent implementations reach semantically equivalent results over the same inputs.

The contracts define inputs, outputs and observable statuses, and the vectors fix cases with an expected outcome. For operations to which idempotency applies, the specification defines the key scope — receiving implementation, authenticated caller, operation — and the request identity, the digest of its BCJ/1 canonical representation: within one scope, an equal identity is a replay and a different one a conflict. The record is retained for a normative minimum of 24 hours after the terminal state, and the effective window is declared.

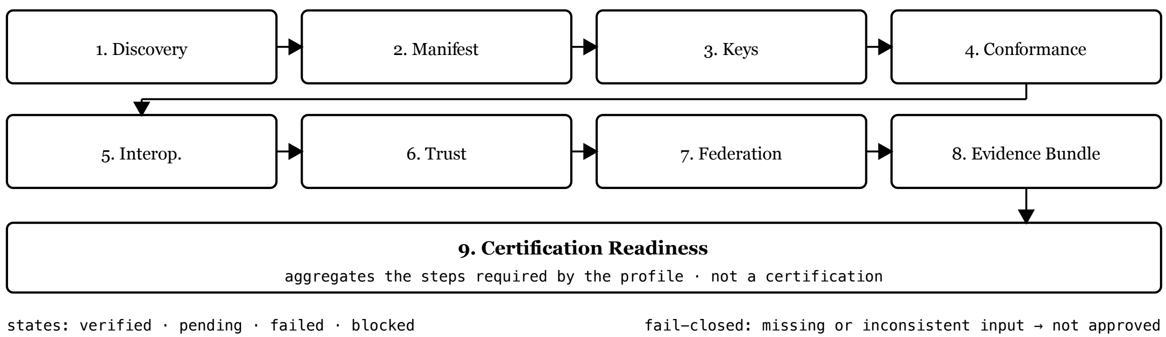

Figure 6. Eight technical steps and one aggregation step, which gathers the steps required by the applicable profile.

The evaluation is directional: B publishes its material, A obtains the artifacts from the canonical origin and evaluates locally. The operational use of the result depends on A's policy.

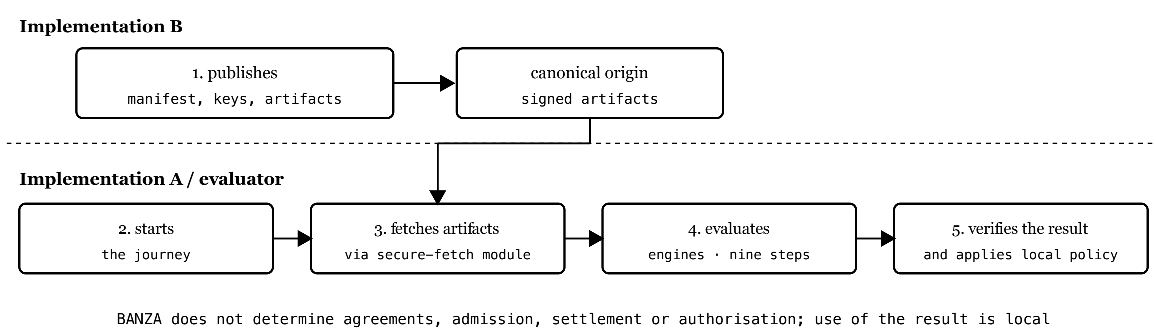

Figure 7. A→B evaluation: B publishes verifiable material, A evaluates locally and can reproduce the result. The evaluation requires no BANZA central intermediary.

7 Evidence

The receipts bind each result to the version, the profile, the artifacts consumed, their digests and the reason codes, allowing the technical basis of the evaluation to be reproduced.

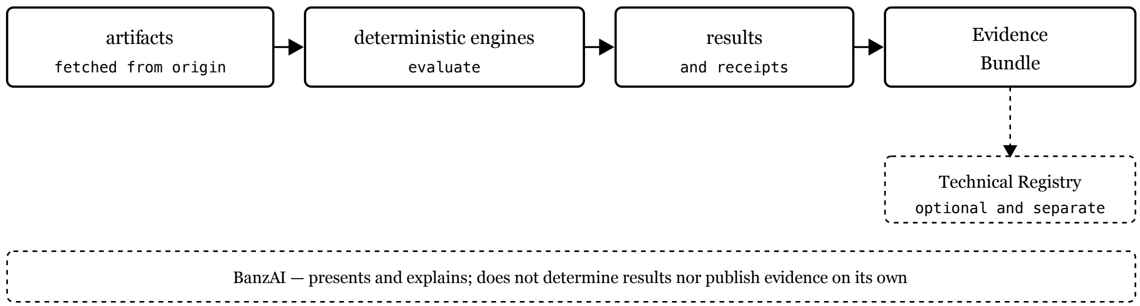

Figure 8. The engines produce results and receipts; the Evidence Bundle gathers verifiable material and publication in the Technical Registry is optional.

A receipt represents the state observed at the instant of evaluation; later changes require a new evaluation. The Evidence Bundle supports the result without conferring operational status: in the Technical Registry, absence is not a prohibition and presence is not an authorisation. Independent verification requires the evidence to suffice for a third party to repeat the applicable steps without any private decision.

The same criterion applies to the development of the protocol. A test is evidence only when it demonstrates the claimed property *for the intended reason*. Each critical property is evaluated across layers distinct by function — from normative authority to representation on the wire, to enforcement by the implementation, to behaviour under failure and attack, and to the completeness of the material needed to implement it without the reference engines. No layer validates itself, and a higher one does not compensate for the failure of a lower one. Categories follow that do not imply one another: *specified*, *implemented*, *internally assured*, *independently implemented* and *operationally demonstrated*.

8 Security

Each threat is met by a mechanism: tampering is detected by signatures and digests; an invalid origin is excluded by resolution from the canonical origin; revoked or expired material is rejected; challenge replay is prevented by single-use challenges; SSRF and DNS rebinding are blocked by hardened remote fetching; absent or inconsistent evidence fails closed. None depends on implicit trust in the origin and none substitutes for the others: the signature establishes authenticity, validity and revocation say whether the material is usable now, and the monotonic state prevents old material, still valid, from replacing more recent material.

The secure-fetch module is the only component that reaches the operator's endpoints. The target is resolved from the canonical origin, never from an arbitrary URL supplied by the caller; it resolves the target from the canonical origin, never from a caller-supplied URL; accepts only HTTPS, blocks private, local and cloud-metadata addresses, connects only to previously validated addresses, follows no redirects, validates TLS and bounds responses.

Resilience is subordinated to security. The failure classes range from the failure of a process, of persisted state or of an external dependency to the unavailability of a peer, network uncertainty and the duplicate delivery that follows from it, failure to publish trust material, and the loss of a Root authority. The intended behaviour is a progression — normal operation, safely degraded operation, localised fail-closed and deterministic recovery —, contained where the dependencies allow. Failing locally is not continuing unsafely: when trust, authorisation, integrity or correctness cannot be established, the correct response is to refuse. *Safety before availability* — no unavailability authorises accepting unsigned material, extending an expired validity or lowering a threshold.

9 Governance

BANZA evolves through public and versioned instruments: RFCs structure proposals, ADRs record architecture decisions. Versioning is explicit — incompatible changes require a new major version, compatible ones enter minor or patch versions — and each implementation declares the version and profile it adopts. Compatibility is owed from the first freeze for external implementation: before it, a correction corrects the version being prepared and the number does not move; after it, a change requires whatever version move compatibility demands. 1.0.0 is still the version being prepared.

Governance determines how the rules evolve without making the maintainer a technical intermediary: implementing a published version is distinct from having authority to change it.

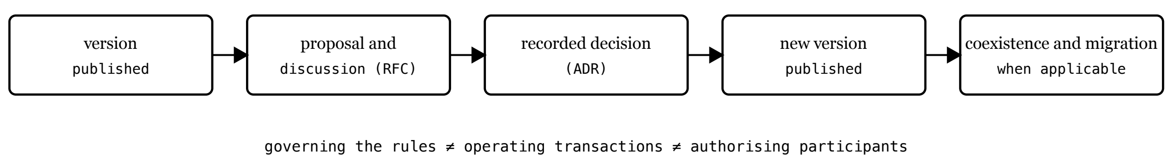

Figure 9. Evolution separates proposal, recorded decision and versioned publication; governing rules is not operating transactions nor authorising participants.

Incompatible changes are announced with a migration path, allowing temporary coexistence; deprecation is explicit and phased.

10 Limitations

BANZA's guarantees are technical, not regulatory: a positive result demonstrates conformance with a public profile, without implying authorisation to operate, admission to a scheme or a commercial agreement.

They apply to what is observed. Reproducibility covers deterministic evaluations over the declared inputs, not the operator's internal controls nor behaviour outside the protocol; and a compromised origin or key is detectable only when it manifests in observable signals or in revocation.

The openness of the specification does not by itself demonstrate the ease of independent implementation — that property depends on the completeness of the contracts and the quality of the vectors — nor does the integration with external infrastructures envisaged by the architecture demonstrate effective access to them, which may depend on interfaces, agreements, scheme rules or authorisation.

Other boundaries follow from the design of this version. Version 1.0.0 does not define redundant distribution of already-signed trust artifacts: published at a single canonical point, the unavailability of that point may prevent a new evaluation — redundancy would address that failure mode, not availability in general, and trust authority and distribution availability are distinct properties. The monotonic state of Section 5 is local, on a first observation staleness is not detectable, and it does not guarantee that several individually valid and fresh artifacts belong to one coherent publication state. With no external profile published, L4 remains unevaluated. The evidence presented is architectural: it demonstrates mechanisms in the reference implementation, without measuring performance, scale, availability in production or adoption.

11 State

As of this edition BANZA remains in pre-production: the Technical Registry contains zero production operators and zero active technical certifications, payments with real money are disabled, and the empty registry is the expected state. The reference implementation, Operator Zero, runs in an isolated test environment with demonstration currency, moves no real funds, remains uncertified and is evaluated by the same contracts as any other.

The normative surface is indexed, and the set required by each profile derives from that indexing. From it, clean-room material was derived for profile L0, without the reference implementation's code; an internal verification — which identified real questions, since corrected — concluded that it is closed upon itself, without depending on the source repository, the decision records or the BanzAI. It is derived and internally validated material: the target of the external trial has not been selected or frozen, and no final trial package has been published.

The completeness of a package and the capability for independent implementation are distinct properties: the first was verified internally, the second was not. Demonstrating it requires an implementation built by a genuinely independent team, from the public specifications and contracts, able to satisfy the applicable vectors; the interoperability between implementations so obtained is a separate experimental step. Neither has begun.

12 Conclusions

This work presented BANZA as an open financial interoperability protocol that separates a common public specification from the implementations and infrastructures that execute it. The aim is not to create a new operational intermediary, but to make public, versioned and verifiable the rules needed for independent implementations to be assessed against the same technical basis.

The main contributions are:

- a public and explicitly indexed normative surface, organised by versions, profiles and contracts, without making the reference implementation the authority of the protocol;
- a model of deterministic and reproducible conformance, based on canonical representation, statuses, reason codes, idempotency, vectors, receipts and evidence bound to the inputs evaluated;
- the separation between protocol, implementation, technical conformance, certification, scheme operation and regulatory authorisation, preventing a decision in one plane from propagating to the others;
- a trust architecture with three authorities and a 2-of-3 threshold, pinned genesis, predecessor-authorised succession, rollback protection and no unilateral recovery;
- a progressive adoption through the L0–L4 profiles, guided by the **BANZA $\mathbf{R^2S^2}$** principles — Robust, Resilient, Secure and Simple — and by the five Architectural Invariants.

These contributions define an architecture and a basis for evaluation, not an operational maturity already demonstrated. BANZA remains in pre-production. The completeness of the L0 material was verified internally, but it remains to be demonstrated that an external team can build a conformant implementation without recourse to the reference code. Interoperability between independent implementations, as well as performance, scale and behaviour under real operational conditions, is additional evidence still to be obtained.

The distinction between specified, implemented, internally assured, independently implemented and operationally demonstrated explicitly bounds what may be claimed at each stage. The next experimental step is therefore to freeze a public L0 target and submit it to a genuinely independent implementation.

In summary, BANZA proposes a public and verifiable basis for financial interoperability without requiring a mandatory implementation, operational authority or central transactional infrastructure.

References

[1] International Organization for Standardization. ISO 20022 — Financial services — Universal financial industry message scheme. ISO. https://www.iso20022.org/

[2] International Organization for Standardization. ISO 8583:2023 — Financial-transaction-card-originated messages — Interchange message specifications. Edition 3. ISO, 2023. https://www.iso.org/standard/79451.html

[3] EMVCo, LLC. EMV QR Code Specification for Payment Systems (EMV QRCPS): Merchant-Presented Mode, Version 1.1. EMVCo, November 2020. https://www.emvco.com/

[4] International Organization for Standardization / International Electrotechnical Commission. ISO/IEC 9646-1:1994 — Information technology — Open Systems Interconnection — Conformance testing methodology and framework — Part 1: General concepts. ISO/IEC, 1994.

[5] National Institute of Standards and Technology (NIST). Secure Hash Standard (SHS). FIPS PUB 180-4. NIST, August 2015. DOI 10.6028/NIST.FIPS.180-4. https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf

[6] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IRTF, January 2017. DOI 10.17487/RFC8032. https://www.rfc-editor.org/rfc/rfc8032

[7] M. Nottingham. Well-Known Uniform Resource Identifiers (URIs). RFC 8615, IETF, May 2019. DOI 10.17487/RFC8615. https://www.rfc-editor.org/rfc/rfc8615

[8] R. Peng. Reproducible Research in Computational Science. Science, 334(6060):1226–1227, 2011. DOI 10.1126/science.1213847. https://doi.org/10.1126/science.1213847

[9] Mojaloop Foundation. Open API for FSP Interoperability (FSPIOP) Specification and Mojaloop open-source platform. https://docs.mojaloop.io/

[10] A. Rundgren, B. Jordan and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, IETF, June 2020. DOI 10.17487/RFC8785. https://www.rfc-editor.org/rfc/rfc8785